

Bataille navale

- Etape 1
- Etape 2
- Etape 3
- Etape 4
- Etape 5
- Etape 6
- Etape 7
- Etape 8
- Conclusion

Introduction

Dans ce TP, l'objectif était de programmer une version simplifiée du jeu de la bataille navale en Java. Pour cela, nous avons suivi différentes étapes permettant d'initialiser les grilles, de placer les pions du joueur et de l'ordinateur, d'afficher les tableaux, puis de gérer les tours de jeu jusqu'à la fin de la partie. Ce TP m'a permis de pratiquer les boucles, les tableaux, les procédures et l'utilisation du hasard.

Étape 1 – Création et initialisation des tableaux

La première étape consiste à créer deux tableaux 5x5 : un pour le joueur et un pour l'ordinateur. Chaque case doit être initialisée à 0 pour indiquer qu'elle est vide.

Pour cela, on déclare deux tableaux à deux dimensions, puis on parcourt chaque ligne et chaque colonne afin d'attribuer la valeur 0 à toutes les cases.

```
int tableauJoueur[][] = new int [5][5];  
int tableauordi[][] = new int [5][5];
```

Cette étape permet de mettre en place la base du jeu et de comprendre comment parcourir un tableau à l'aide de boucles imbriquées.

```
for (int ligne = 0; ligne < 5; ligne++) {  
    for (int col = 0; col < 5; col++) {  
        champJoueur[ligne][col] = 0;  
        champBot[ligne][col] = 0;  
    }  
}
```

Étape 2 – Placement des pions du joueur

Ensuite, le joueur doit placer ses cinq pions.

Le programme lui demande de saisir une ligne et une colonne comprises entre 0 et 4. Plusieurs vérifications sont nécessaires :

- Les valeurs doivent bien être dans l'intervalle autorisé.

- La case ne doit pas déjà contenir un pion.

Tant que l'un des critères n'est pas respecté, le joueur doit recommencer.

Une fois qu'une position correcte est donnée, la case correspondante du tableau du joueur prend la valeur 1.

Cette partie demande l'utilisation de conditions, d'un compteur et de saisies répétées via un scanner.

```
Scanner entree = new Scanner(System.in);

int pionsPlaces = 0;

while (pionsPlaces < 5) {

    System.out.println("Choisissez une ligne (0-4) : ");
    int ligneChoisie = entree.nextInt();

    System.out.println("Choisissez une colonne (0-4) : ");
    int colonneChoisie = entree.nextInt();

    if (ligneChoisie < 0 || ligneChoisie > 4 || colonneChoisie < 0 || colonneChoisie > 4) {
        System.out.println("Coordonnées invalides !");
    }
    else if (champJoueur[ligneChoisie][colonneChoisie] == 1) {
        System.out.println("Cette case est déjà prise !");
    }
    else {
        champJoueur[ligneChoisie][colonneChoisie] = 1;
        pionsPlaces++;
        System.out.println("Bateau placé !");
    }
}
```

Étape 3 – Placement des pions de l'ordinateur

Le placement des pions de l'ordinateur fonctionne sur le même principe, mais cette fois la ligne et la colonne sont choisies aléatoirement grâce à `Math.random()`.

On génère une position, puis on vérifie si la case est encore libre.

Si elle est déjà occupée, un nouveau tirage aléatoire est effectué jusqu'à trouver une case à 0.

Lorsque la case est valide, elle devient 1.

Cette étape met en pratique l'utilisation du hasard et de la boucle `do...while`.

```
int pionsBotPlaces = 0;

while (pionsBotPlaces < 5) {

    int ligneAlea = (int)(Math.random() * 5);
    int colAlea = (int)(Math.random() * 5);

    if (champBot[ligneAlea][colAlea] == 0) {
        champBot[ligneAlea][colAlea] = 1;
        pionsBotPlaces++;
    }
}
```

Étape 4 – Procédure d’affichage du tableau du joueur

Pour faciliter la visualisation des pions du joueur, une procédure d’affichage est créée. Elle affiche :

- les numéros de colonnes,
- puis chaque ligne,
- en remplaçant les valeurs :
 - 1 → un pion représenté par « O »
 - 0 → une case vide représentée par « ~ »

Cette procédure prend en paramètre le tableau et le nombre de cases.

```

static void afficherGrille(int[][] grille, int taille) {
    System.out.print(" ");
    for (int c = 0; c < taille; c++) {
        System.out.print(c + " ");
    }
    System.out.println();

    for (int ligne = 0; ligne < taille; ligne++) {
        System.out.print(ligne + " ");
        for (int col = 0; col < taille; col++) {
            if (grille[ligne][col] == 1) {
                System.out.print("O ");
            } else {
                System.out.print("~ ");
            }
        }
        System.out.println();
    }
}

```

Étape 5 – Affichage du tableau caché (pour l'ordinateur)

Afin que le joueur ne voie pas les positions des pions adverses, on crée une procédure d'affichage "caché".

Cette fois-ci :

- les cases non découvertes s'affichent sous forme de ?
- une case touchée avec un pion apparaît avec O
- une case révélée sans pion apparaît avec X

Ces nouveaux états (0, 2, 3) permettent de distinguer les différents types de découvertes.

```

static void afficherMasque(int[][] grille, int taille) {
    System.out.print(" ");
    for (int c = 0; c < taille; c++) {
        System.out.print(c + " ");
    }
    System.out.println();

    for (int ligne = 0; ligne < taille; ligne++) {
        System.out.print(ligne + " ");
        for (int col = 0; col < taille; col++) {
            if (grille[ligne][col] == 0) {
                System.out.print("? ");
            } else if (grille[ligne][col] == 2) {
                System.out.print("O ");
            } else if (grille[ligne][col] == 3) {
                System.out.print("X ");
            }
        }
        System.out.println();
    }
}

```

Étape 6 – Tour du joueur

Lors du tour du joueur, celui-ci entre une ligne et une colonne pour tenter de toucher un pion de l'ordinateur.

On vérifie alors :

- s'il s'agit d'une case qui contient un pion ennemi, dans ce cas elle devient 2 et un compteur de pions trouvés augmente,
- sinon, la case devient 3, indiquant un tir dans le vide.

Cette étape peut être mise dans une fonction qui renvoie 1 si un pion est touché, 0 sinon.

```
static int tirJoueur(int[][] champBot, int ligne, int colonne) {
    if (champBot[ligne][colonne] == 1) {
        champBot[ligne][colonne] = 2;
        return 1;
    } else if (champBot[ligne][colonne] == 0) {
        champBot[ligne][colonne] = 3;
    }

    return 0;
}
```

Étape 7 – Tour de l'ordinateur

Le tour de l'ordinateur suit le même principe que celui du joueur, mais les positions sont choisies aléatoirement.

L'ordinateur tire sur la grille du joueur :

- si un pion est touché → la valeur passe à 2 et le compteur de l'ordinateur augmente,
- sinon → la valeur passe à 3.

Cette partie reprend les mêmes vérifications que le tour du joueur mais en automatisé.

```
static int tirAdversaire(int[][] champJoueur) {
    int ligneAlea, colAlea;

    do {
        ligneAlea = (int)(Math.random() * 5);
        colAlea = (int)(Math.random() * 5);
    } while (champJoueur[ligneAlea][colAlea] == 2 || champJoueur[ligneAlea][colAlea] == 3);

    if (champJoueur[ligneAlea][colAlea] == 1) {
        champJoueur[ligneAlea][colAlea] = 2;
        return 1;
    } else {
        champJoueur[ligneAlea][colAlea] = 3;
        return 0;
    }
}
```

Étape 8 – Fin de partie et possibilité de rejouer

La partie s'arrête lorsqu'un joueur atteint 5 pions trouvés.

À ce moment-là, un message affiche le vainqueur.

Enfin, le programme propose au joueur de recommencer une nouvelle partie.

Pour pouvoir rejouer, l'ensemble du programme est inclus dans une boucle qui ne s'arrête que si le joueur choisit de quitter.

```
if (pionsTrouvesJoueur == 5) {  
    System.out.println("Vous avez gagné !");  
} else {  
    System.out.println("L'ordinateur a gagné.");  
}  
  
System.out.println("Voulez-vous rejouer ? (1=oui / 0=non)");  
int rejouer = entree.nextInt();
```

Conclusion

Ce TP m'a permis de mieux comprendre la manipulation des tableaux à deux dimensions, l'usage des boucles et du hasard, ainsi que l'intérêt des procédures pour organiser le code. Le début du TP était assez simple, mais les dernières étapes devenaient plus complexes car plusieurs éléments devaient interagir ensemble. Malgré cela, l'exercice était formateur et m'a aidé à progresser dans la structuration d'un programme Java.